Deadlock Pseudo Code:

deadlock.py > ...

```python
n = 5
r = 3

# Allocation
alloc = [
    [0, 0, 1],  # P0
    [3, 0, 0],  # P1
    [1, 0, 1],  # P2
    [2, 3, 2],  # P3
    [0, 0, 3]   # P4
]
# Max
maxm = [
    [7, 6, 3],  # P0
    [3, 2, 2],  # P1
    [8, 0, 2],  # P2
    [2, 1, 2],  # P3
    [5, 2, 3]   # P4
]
# Available
avail = [2, 3, 2]

f = [0] * n         # finished
ans = [0] * n       # safe sequence
ind = 0
```

```python
# Need
need = [[0] * r for _ in range(n)]
for i in range(n):
    for j in range(r):
        need[i][j] = maxm[i][j] - alloc[i][j]

# Banker's Algorithm
for _ in range(n):
    for i in range(n):
        if f[i] == 0:
            flag = 0
            for j in range(r):
                if need[i][j] > avail[j]:
                    flag = 1
                    break

            if flag == 0:
                ans[ind] = i
                ind += 1
                for y in range(r):
                    avail[y] += alloc[i][y]
                f[i] = 1

print("The SAFE Sequence is as follows")
for i in range(n - 1):
    print(f"P{ans[i]} ->", end=" ")
print(f"P{ans[n - 1]}")
```

Input/Output:

```
D:\Academic\3rd Year 1st Semester\z Ou
The SAFE Sequence is as follows
P1 -> P3 -> P4 -> P0 -> P2
```